

✓ Part 2: Preprocessing

Fake News Detection — Data Preprocessing

This notebook contains: Data loading + Train/Val/Test split, TF-IDF, tokenization for transformers.

✓ Step 1: Imports

```

# Install (run once): pip install transformers datasets scikit-learn

import pandas as pd
import numpy as np
import time
import matplotlib.pyplot as plt
import seaborn as sns
import torch
import re
from collections import Counter

from sklearn.model_selection import train_test_split, GridSearchCV,
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, classification_report,
                             precision_recall_fscore_support, roc_auc_score)

# scipy.stats does NOT provide mcnemar; use statsmodels instead
from statsmodels.stats.contingency_tables import mcnemar

from datasets import Dataset, DatasetDict
from transformers import AutoTokenizer, AutoModelForSequenceClassification

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
torch.manual_seed(RANDOM_STATE)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(RANDOM_STATE)

try:
    plt.style.use('seaborn-v0_8-whitegrid')
except:
    plt.style.use('ggplot')

print(f"PyTorch: {torch.__version__}")
print(f"Device: {torch.cuda.get_device_name(0)} if torch.cuda.is_available()")

PyTorch: 2.10.0+cpu
Device: CPU

```

✓ Step 2: Load Dataset (DO NOT MODIFY)

```
url_fake = "https://raw.githubusercontent.com/laxmimerit/fake-real-n
url_true = "https://raw.githubusercontent.com/laxmimerit/fake-real-n

print("Downloading and Reconstructing Dataset...")

df_fake = pd.read_csv(url_fake)
df_true = pd.read_csv(url_true)
df_fake['label'] = 1
df_true['label'] = 0

df_combined = pd.concat([df_fake, df_true], axis=0).sample(frac=1, r
df_combined['text'] = df_combined['title'] + " " + df_combined['text
df_combined = df_combined[['text', 'label']]

print(f"✓ Dataset: {len(df_combined)} samples")

train_df, test_df = train_test_split(df_combined, test_size=0.2, ran
ds = DatasetDict({'train': Dataset.from_pandas(train_df), 'test': Da

print(f"Train: {len(ds['train'])}, Test: {len(ds['test'])}")
```

```
Downloading and Reconstructing Dataset...
✓ Dataset: 44898 samples
Train: 35918, Test: 8980
```

✓ Step 4: Preprocessing

```

X_train_text = list(ds['train']['text'])
y_train = np.array(ds['train']['label'])
X_test_text = list(ds['test']['text'])
y_test = np.array(ds['test']['label'])

# Train/Validation split (90/10) for model selection and early stop
X_tr, X_val, y_tr, y_val = train_test_split(X_train_text, y_train,
print(f"Train: {len(X_tr)}, Val: {len(X_val)}, Test: {len(X_test_te

tfidf = TfidfVectorizer(max_features=5000, stop_words='english', ng
X_train_tfidf = tfidf.fit_transform(X_tr)
X_val_tfidf = tfidf.transform(X_val)
X_test_tfidf = tfidf.transform(X_test_text)
print(f"TF-IDF: {X_train_tfidf.shape}")

# For transformers: create train+val datasets
train_val_df = pd.DataFrame({'text': X_tr + X_val, 'label': list(y_
train_small = train_val_df.iloc[:len(X_tr)]
val_small = train_val_df.iloc[len(X_tr):]
ds_train_val = DatasetDict({
    'train': Dataset.from_pandas(train_small.reset_index(drop=True)
    'validation': Dataset.from_pandas(val_small.reset_index(drop=Tr
    'test': ds['test']
}))

checkpoint = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
def tok_fn(ex): return tokenizer(ex['text'], padding='max_length',
tokenized_ds = ds_train_val.map(tok_fn, batched=True)
tokenized_ds.set_format('torch', columns=['input_ids', 'attention_ma
print("✓ Preprocessing done")

```

Train: 32326, Val: 3592, Test: 8980

TF-IDF: (32326, 5000)

Map: 100%

32326/32326 [01:04<00:00, 488.84 examples/
s]

Map: 100%

3592/3592 [00:06<00:00, 565.99 examples/
s]

